

Project: OOP Wrapper

Objective:

The goal of this project is to build an Employee Management System that utilizes various Object-Oriented Programming (OOP) concepts like classes, inheritance, encapsulation, method overloading, method overriding, and more. The system will model employee data and operations related to adding, updating, and removing employees, as well as tracking their roles (e.g., Manager, Developer).

Project Structure & Requirements:

Employee Class (Base Class):

- The Employee class should have attributes such as employee_id, name, age, and salary.
- Provide getter and setter methods for salary and employee_id.
- Define a constructor to initialize these attributes and a destructor to clean up resources.
- Use self to refer to instance attributes.

Manager Class (Derived Class):

- Inherits from Employee.
- Additional Attribute: Adds a new attribute department (e.g., HR, Finance).
- Override display() to include department information.

Developer Class (Derived Class):

- Inherits from Employee.
- Additional Attribute: Adds a new attribute `programming_language` (e.g., Python, Java).
- Override `display()` to include programming language information.

Method Overloading:

- Provide different constructors in the Employee class to allow multiple ways of creating employee objects.

Encapsulation:

- Encapsulate sensitive data like salary and employee_id by making them private and providing getter and setter methods.

Inheritance and Method Overriding:

- Use inheritance to create specialized classes like Manager and Developer.
- Override the `display()` method to include specific details.

Use of `super()` and `issubclass()`:

- Use `super()` to call the parent class methods.
- Use `issubclass()` to check if a class is a subclass of Employee class.

User Interface (UI):

- Create a menu-driven interface to let users choose from different options.
- Allow users to exit the program from the main menu.

Example Console Interaction:

--- Python OOP Project: Employee Management System ---

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager
4. Show Details
5. Exit

Enter your choice: 1

Enter Name: John Doe

Enter Age: 30

Person created with name: John Doe and age: 30.

--- Choose another operation ---

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager
4. Show Details
5. Exit

Enter your choice: 2

Enter Name: Jane Smith

Enter Age: 28

Enter Employee ID: E123

Enter Salary: 50000

Employee created with name: Jane Smith, age: 28, ID: E123, and salary: \$50000.0.

--- Choose another operation ---

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager
4. Show Details
5. Exit

Enter your choice: 3

Enter Name: Alice Johnson

Enter Age: 40

Enter Employee ID: M456

Enter Salary: 80000

Enter Department: Sales

Manager created with name: Alice Johnson, age: 40, ID: M456, salary: \$80000.0, and department: Sales.

--- Choose another operation ---

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager
4. Show Details
5. Exit

Enter your choice: 4

Choose details to show:

1. Person
2. Employee
3. Manager

Enter your choice: 2

Employee Details:

Name: Jane Smith

Age: 28

Employee ID: E123

Salary: \$50000.0

--- Choose another operation ---

Choose an operation:

1. Create a Person
2. Create an Employee
3. Create a Manager
4. Show Details
5. Exit

Enter your choice: 5

Exiting the system. All resources have been freed.

Goodbye!

Instructions:

Task Completion: Ensure that you attempt all the assigned tasks given to you as part of the exam. Complete each task to the best of your ability, following the instructions provided.

Assumptions: Make suitable assumptions wherever necessary, based on the requirements and instructions provided. Document any assumptions made in your project documentation or README.md file.

GitHub Repository: Create a GitHub repository to host your project. Upload your project files, including source code, and documentation to the repository. Ensure that you provide a clear and descriptive README.md file.

No Copying: Do not copy code or any other content from your classmates or any other source. Plagiarism is strictly prohibited and can result in severe consequences, including academic penalties. Ensure that all the code and content in your project are original and properly attributed to the appropriate sources, if applicable.

Submission: Once you have completed your project and uploaded it to your GitHub repository, submit the GitHub repository link to your instructor or as instructed. Double-check that your repository is properly organized and includes all the required files as per the instructions provided.

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!